# Java 热门面试题 60 道速记通关版

演示用题库 · 题干与答案按字号台阶排版，供 PDF 解析演示

### HashMap 底层结构是什么?

数组加链表加红黑树。
链表长度超过 8 且数组长度超过 64 时转红黑树，退化为 6 时转回链表。

### HashMap 为什么线程不安全?

多线程同时扩容会丢数据。
JDK 7 的头插法迁移还可能造成链表死循环。
并发场景请用 ConcurrentHashMap。

### ConcurrentHashMap 如何保证并发?

JDK 7 用分段锁，段之间互不干扰。
JDK 8 改为 CAS 加 synchronized 锁链表头节点。
锁粒度更细，读操作基本无锁。

### synchronized 和 ReentrantLock 区别?

synchronized 是 JVM 关键字，自动加解锁。
ReentrantLock 是 API，可中断、可公平、可限时尝试加锁。

### volatile 能保证原子性吗?

不能，它只保证可见性与禁止指令重排。
计数等复合操作仍要锁或原子类。

### JVM 内存结构包含哪些区域?

堆、方法区、虚拟机栈、本地方法栈、程序计数器。
堆是垃圾回收主战场，栈随线程创建而生、销毁而灭。

### 对象什么时候进入老年代?

大对象直接进入老年代。
长期存活对象年龄达到 15 岁晋升。
动态年龄判定达标时同批对象一起晋升。

### GC Root 有哪些?

虚拟机栈中的引用、方法区静态属性、常量池引用。
本地方法栈里 JNI 引用的对象也是根。

### CMS 和 G1 的区别?

CMS 追求低延迟，但标记清除会产生内存碎片。
G1 面向服务端，按 Region 回收，停顿时间可预测。

### 类加载过程分几步?

加载、验证、准备、解析、初始化五步。
双亲委派发生在加载阶段的类加载器协作上。

### 双亲委派模型是什么?

加载器先委派父加载器尝试，父不行才自己加载。
好处是核心类不会被自定义实现篡改。

### 如何打破双亲委派?

自定义加载器重写 loadClass 即可绕过委派。
JDBC 等服务接口用的是线程上下文类加载器这条路。

### 什么是内存泄漏?

对象已无用处却仍被引用，GC 无法回收。

常见于静态集合、未注销的监听器、忘记关闭的连接。

### 线程池核心参数有哪些?

核心线程数、最大线程数、空闲存活时间、工作队列。
再加拒绝策略与线程工厂，共六件套。

### 线程池拒绝策略有几种?

中止、直接丢弃、丢弃最老、调用者执行四种。
生产环境常用调用者执行来兜底降压。

### Executors 有什么问题?

默认使用无界队列，请求堆积会拖垮内存。
应手动指定参数并改用有界队列。

### ThreadLocal 原理与坑?

每个线程持有一份变量副本，天然隔离。
用完后必须 remove，否则线程池复用会内存泄漏。

### AQS 是什么?

抽象队列同步器，用 volatile 状态量加 CLH 队列。
ReentrantLock、Semaphore、CountDownLatch 都基于它。

### synchronized 锁升级过程?

无锁、偏向锁、轻量级锁、重量级锁逐级晋升。
只有竞争出现才升级，各状态撤销都有开销。
JDK 15 起偏向锁已默认禁用，别照背旧八股。

### CAS 是什么？有什么问题?

比较并交换，靠一条 CPU 原子指令实现的乐观锁。
存在 ABA 问题，可用版本号方案解决。
自旋忙等会消耗 CPU，适合短临界区。

### 死锁的四个条件?

互斥、请求保持、不可剥夺、循环等待。
破坏任一条件即可避免，比如全局按序加锁。
排查用 jstack 找 blocked 线程与锁持有关系。

### Java 内存模型是什么?

主内存与工作内存之间的抽象约定。
通过 happens-before 规则保证可见性与有序性。

### 什么是 happens-before?

操作之间的偏序关系，前一个操作对后一个可见。
常见规则：程序次序、volatile 写读、锁释放获取。

### 重载与重写的区别?

重载看参数列表，编译期绑定。
重写看方法签名，运行期动态分派，返回类型要兼容。

### String 为什么不可变?

字符数组是 final 的，且不提供任何修改方法。
好处是哈希可缓存、天生线程安全，适合做 Map 的键。

### String、StringBuilder、StringBuffer?

String 不可变，拼接会产生新对象。
StringBuilder 非线程安全但速度最快。
StringBuffer 方法都加了锁，线程安全。

### equals 与 hashCode 约定?

相等则哈希码必须相同，哈希相同不一定相等。
重写 equals 必须同时重写 hashCode，否则哈希容器失效。

### 深拷贝与浅拷贝区别?

浅拷贝只复制引用，新旧对象共享子对象。
深拷贝递归复制所有层级，彼此完全独立。

### 泛型擦除是什么?

编译期泛型信息被擦除为上限类型。
运行时可借助反射读取签名上的残留痕迹。

### 反射破坏单例怎么办?

反射可以调用私有构造器拿到第二个实例。
枚举单例由 JVM 保证，反射也无法破坏。

### 泛型上下界 PECS 原则?

生产者用 extends，消费者用 super。
在集合读写之间取得灵活平衡。

### ArrayList 扩容机制?

默认容量 10，满了按 1.5 倍扩容。
底层用 Arrays.copyOf 拷贝，频繁扩容有开销。

### ArrayList 与 LinkedList 区别?

ArrayList 基于数组，随机访问快。
LinkedList 基于链表，首尾插删快但缓存不友好。

### fail-fast 是什么?

迭代期间结构被修改会抛并发修改异常。
可用迭代器自身删除，或换 CopyOnWriteArrayList。

### HashMap 初始容量怎么设?

按预期元素量除以负载因子 0.75 估算。
避免边用边扩容，初始值会取到 2 的幂。

### 红黑树有什么特性?

根与叶子必黑、红结点的子结点必黑、黑高相等。
插入删除最多三次旋转即可恢复平衡。

### BIO、NIO、AIO 区别?

BIO 一个连接一个线程，阻塞等待。
NIO 多路复用，单线程管理多条通道。
AIO 异步回调，操作完成才通知。

### Netty 为什么快?

零拷贝、内存池化、无锁化设计与 Reactor 模型。
还修复了 NIO 著名的 epoll 空轮询问题。

### select、poll、epoll 区别?

select 有连接数上限且要反复拷贝描述符。
poll 去掉了上限，但仍要遍历轮询。
epoll 事件驱动，只返回就绪的那些。

### TCP 三次握手过程?

客户端发 SYN，服务端回 SYN 加 ACK，客户端再回 ACK。
第三次握手即可携带数据，省一次往返。

### 为什么握手是三次不是两次?

三次才能让双方都确认对方收发正常。
同时完成初始序列号的同步，防止历史重复连接。

### TCP 四次挥手为什么?

关闭是双向的，要各自发一次 FIN。
服务端收到 FIN 后可能还有数据要发，故 ACK 与 FIN 分两步。

### TIME_WAIT 是什么?

主动关闭方等待两个最长报文寿命的时间。
确保最后的 ACK 被收到，并让旧报文自然消散。

### TCP 拥塞控制有哪些?

慢启动、拥塞避免、快重传、快恢复四步。
配合滑动窗口与接收缓冲区共同调速。

### HTTP 1.1 与 2.0 区别?

2.0 改为二进制分帧，支持多路复用。
还有头部压缩与服务端推送，解决了队头阻塞。

### HTTPS 握手做了什么?

验证证书、协商密码套件、ECDHE 交换密钥。
之后全程用对称加密通信。

### Cookie 与 Session 区别?

Cookie 存在客户端，体积受限且有泄露风险。
Session 存服务端，客户端只带一个会话 ID。

### GET 与 POST 的区别?

GET 参数在地址栏，可缓存且幂等。
POST 参数在请求体，适合写操作与敏感数据。

### 什么是 XSS? 如何防?

攻击者注入脚本在他人浏览器里执行。
防范靠输出转义、内容安全策略与 HttpOnly Cookie。

### CSRF 怎么防?

核心是确认请求来自真实用户而非伪造站点。
手段有校验来源、一次性 Token 与 SameSite Cookie。

### SQL 注入如何防?

一律用预编译占位符，绝不拼接 SQL。
再配合输入校验与数据库最小权限。

### 聚簇索引与非聚簇索引?

InnoDB 聚簇索引叶子结点就是数据行。
MyISAM 非聚簇索引叶子存的是地址，需二次查找。

### 什么是最左前缀匹配?

联合索引从最左列开始，遇到范围查询即停。
索引列顺序设计比索引数量更关键。

### 事务的 ACID 与隔离级别?

原子性、一致性、隔离性、持久性四大特性。
隔离级别有读未提交、读已提交、可重复读、串行化。

### MVCC 如何实现?

三件套：隐式版本列、undo 版本链、ReadView。
快照读靠可见性判断，读写互不阻塞。
当前读仍走锁；可重复读下首次 ReadView 定快照。

### 什么是间隙锁?

锁住记录之间的开区间，防止插入造成幻读。
与 next-key lock 配合，在可重复读级别生效。
代价是锁范围扩大易死锁，事务宜短。

### 慢 SQL 怎么排查?

先开慢日志定位，再 explain 看执行类型。
重点消灭全表扫描与文件排序，索引要建在刀刃上。

### 覆盖索引是什么?

查询列全部在索引上，无需回表查数据行。
能显著减少随机 IO，是性价比最高的优化。

### Redis 为什么快?

纯内存操作，单线程避免上下文切换。
加上 IO 多路复用与高效的数据结构。

### Redis 持久化有几种?

RDB 定时快照与 AOF 命令日志两种。
混合模式兼顾速度与安全，按业务容忍度选择。